Atividade Prática - Aula 02

Trilhas API (Módulo de Busca) + Frontend (Camada de Comunicação com a API)

Baseado em: roteiro da **Aula 02 — trilha API (Shortz-App API)** e roteiro da **Aula 02 — trilha Frontend (Shortz-App)**

Aplicação: este enunciado vale para **qualquer** projeto sorteado na *Lista de Projetos para Programação Web* — use a mesma ficha de preparação que você preencheu na Atividade da Aula 01 (nome do projeto, entidade principal, portas). Não é preciso recriá-la.

Pré-requisito: Atividade da Aula 01 concluída nas duas trilhas — API rodando com `middlewares/apiResponse.js`, `routes/index.js` (`GET /api`) e CORS configurado; front com Vue Router, layout base e a Landing Page consumindo `GET /api` via `fetch`.

> ⚠️ **Ordem de execução:** assim como na Aula 01, faça a **Parte A (API) antes da Parte B (Frontend)** — o teste de ponta a ponta do Bloco final da trilha de front só funciona com a rota de busca do seu back-end já respondendo.

1. Objetivo

Ao final desta atividade, você deve ter, no seu próprio projeto:

Do lado do back-end (API):

- O primeiro **módulo de domínio** da sua API (`modules/search/`), seguindo o padrão de camadas **Route** → **Controller** → **Service** que vai se repetir em todos os módulos futuros do semestre;
- A rota pública `GET /api/search?q=...` respondendo no formato `{ success, message, data: { query, <sua-entidade-no-plural>, users } }`, com listas **vazias** (ainda não há banco de dados);
- `app.js` atualizado com o padrão de registro de módulo (`require` + `app.use('/api', ...)`) que vai se repetir a cada módulo novo dali em diante.

Do lado do front-end:

- Uma **instância única e configurada do Axios** (`src/services/api.js`), com `baseURL` vinda do `.env` e um **interceptor de resposta** que normaliza qualquer erro (validação, autenticação ou rede) no mesmo formato `{ message, errors, status }`;

- Três arquivos de service: `authService.js` (ainda não usado — pronto para daqui a duas/três aulas), `searchService.js` e `systemService.js`;
- A Landing Page da Aula 01 **refatorada** para consumir `systemService` em vez do `fetch` cru;
- Essa camada inteira **validada de ponta a ponta**, com uma chamada real à rota `GET /search` da sua própria API e com o cenário de erro de rede provocado de propósito.

Você também deve ser capaz de **explicar com suas palavras**:

- Por que a lógica de negócio de uma rota é separada em três arquivos (Route, Controller, Service) mesmo quando ela ainda não faz nada de complexo;
- O que é um **interceptor** do Axios, e por que ele evita que cada tela trate erro do seu próprio jeito;
- A diferença entre o `.data` do Axios (removido automaticamente pelo interceptor) e o `data` do envelope da própria API (`{ success, message, data }`).

2. Antes de começar

- Recupere a sua **ficha de preparação** da Atividade da Aula 01 (nome do projeto, entidade principal, portas da API e do front). Você vai reutilizar exatamente os mesmos valores hoje.
- Confirme que a sua API e o seu front da Aula 01 ainda sobem sem erro (`npm run dev` nos dois projetos) antes de começar qualquer coisa nova.

2.1. Uma decisão a tomar antes de começar: o nome do seu campo de busca

Na Shortz-App, a busca devolve `{ query, videos, users }` — `videos` porque a entidade principal do projeto é o vídeo. **No seu projeto, troque `videos` pelo nome (no plural, em `camelCase`) da entidade principal que você registrou na ficha da Aula 01** — por exemplo: `podcasts`, `recipes`, `workouts`, `posts`, `events`, `books`, `songs`, `challenges`, `photos`, `quizzes`, `tutorials`. O campo `users` permanece igual em todos os projetos, já que todo sistema do semestre tem usuários.

> ⚠️ Anote o nome escolhido, acrescentando uma linha à ficha criada na atividade anterior, porque ele vai aparecer em **três lugares diferentes** nesta atividade (Etapa 3 da Parte A, e nos Blocos 3 e 5 da Parte B) — e os três precisam usar exatamente o mesmo nome, com a mesma grafia:

Dado	Seu valor
Nome do campo de busca da sua entidade principal (plural, camelCase)	*(ex.: `podcasts`, `recipes` ...)*

PARTE A — Backend: o primeiro módulo de domínio

3. Contextualização rápida

Nesta atividade a API deve ganhar **um módulo pequeno e "de mentira"** — sem banco de dados ainda, só para fixar, no caso mais simples possível, o padrão de camadas que **toda** rota da sua API vai seguir dali para frente:

- **Route** (`*Routes.js`) → só declara "qual verbo + qual caminho aciona qual função do controller". Sem lógica.
- **Controller** (`*Controller.js`) → lê os parâmetros da requisição, chama o service, formata a resposta com o helper `success` / `error` da Aula 01. Não sabe **como** o dado é obtido.
- **Service** (`*Service.js`) → contém a lógica de negócio e o acesso a dados (hoje, nenhum acesso real — só um valor fixo). É a camada que mais muda ao longo do semestre.

Por que separar isso mesmo numa rota tão simples? Porque o **contrato** que o Controller expõe para fora não muda quando o Service evoluir. Hoje a busca devolve listas vazias fixas; quando o banco de dados entrar em cena (na trilha da Aula 13), só o `searchService.js` da API muda — a rota, o controller e o `searchService.js` do **front** continuam exatamente iguais. É a prova prática de por que camadas separadas importam.

4. Etapa 1 — Criando o módulo `search`

```
mkdir modules\search
```

Isso preenche, pela primeira vez, a pasta `modules/` que ficou vazia na Aula 01 — o primeiro módulo de domínio da sua API nasce aqui.

Crie `modules/search/searchService.js` (o código é o mesmo para qualquer projeto — este arquivo ainda não conhece nada do seu domínio):

```
/**
 * Camada de acesso a dados do módulo de busca.
 *
 * Hoje (Aula 02) ainda não existe conexão com banco de dados — por isso
 * esta função devolve sempre listas vazias. A partir da trilha da Aula 13,
 * este é o único arquivo que vai mudar para consultar o banco de verdade
 * por itens e usuários que combinem com o termo buscado.
 */

function globalSearch(query) {
    return { <sua-entidade-no-plural>: [], users: [] };
}
```

```
module.exports = { globalSearch };
```

> ⚠️ Troque `<sua-entidade-no-plural>` pelo valor que você anotou na Seção 2.1 — inclusive na declaração do objeto (`{ podcasts: [], users: [] }`, por exemplo).

Checklist desta etapa:

- [ ] Pasta `modules/search/` criada
- [ ] `searchService.js` criado, com o nome da sua entidade no plural (não `videos`, a menos que essa seja mesmo a sua entidade)

5. Etapa 2 — Criando o controller

Crie `modules/search/searchController.js`:

```
const searchService = require('./searchService');
const { success } = require('../../middlewares/apiResponse');

// GET /api/search?q=...
exports.search = (req, res) => {
    const query = req.query.q || '';
    let searchResults = { <sua-entidade-no-plural>: [], users: [] };

    if (query) {
        searchResults = searchService.globalSearch(query);
    }

    return success(res, { query, ...searchResults });
};
```

> ⚠️ **Atenção ao nome do parâmetro:** `req.query.q` lê o parâmetro `q` da *query string* (a parte da URL depois do `?`). O nome `q` **precisa bater exatamente** com o que o `searchService.js` do seu **front** vai enviar (Bloco 3.2 da Parte B desta atividade, mais adiante). Se esse nome divergir de um lado para o outro, a busca "funciona" sem erro nenhum, só que sempre devolve vazio — um dos bugs mais silenciosos de uma API, exatamente por não gerar exceção nenhuma.

Checklist desta etapa:

- [ ] `searchController.js` criado, usando `exports.search = ...` (não `module.exports = ...`)
- [ ] O objeto retornado usa o nome da sua entidade no plural, igual ao definido no service

6. Etapa 3 — Criando a rota

Crie `modules/search/searchRoutes.js`:

```
const express = require('express');
const router = express.Router();
const searchController = require('./searchController');

// Busca global. Rota pública.
router.get('/search', searchController.search);

module.exports = router;
```

Note que passamos a **referência** da função (`searchController.search`), sem parênteses — parênteses executariam a função imediatamente ao carregar o arquivo, em vez de deixar o Express chamá-la a cada requisição. E, como a rota `GET /api` , esta também é **pública** — qualquer pessoa, logada ou não, pode buscar.

Checklist desta etapa:

- [ ] `searchRoutes.js` criado, com `router.get('/search', searchController.search)`

7. Etapa 4 — Registrando o módulo em `app.js`

Edite `app.js` , adicionando o novo router junto aos demais:

```
var indexRouter = require('./routes/index');
var searchRoutes = require('./modules/search/searchRoutes');   // ADICIONE ESTA LINHA

// ...

app.use('/api', indexRouter);
app.use('/api', searchRoutes);   // ADICIONE ESTA LINHA
```

Esse é o padrão que vai se repetir a cada módulo novo daqui para frente: uma linha de `require` perto do topo, uma linha de `app.use('/api', ...)` perto da montagem das rotas — sempre nessa ordem. A ordem **entre módulos** não importa (diferente da ordem entre *middlewares*, como o CORS, que importa muito); o Express casa cada requisição com a primeira rota que bater caminho e verbo, e `/search` não conflita com `/` .

Confirme, no terminal, que o `nodemon` reiniciou sem erro de sintaxe.

Checklist desta etapa:

- [ ] `searchRoutes` importado no topo de `app.js`
- [ ] `app.use('/api', searchRoutes)` adicionado
- [ ] API reiniciou sem erros

8. Etapa 5 — Testando isoladamente

```
# Busca com termo
curl "http://localhost:3000/api/search?q=teste"
```

Deve responder (com o nome da sua entidade no lugar de `videos`):

```
{"success":true,"message":null,"data":{"query":"teste","<sua-entidade-no-plural>":[],"user
```

```
# Busca sem termo
curl "http://localhost:3000/api/search"


# Confirme que GET /api continua funcionando (nada quebrou)
curl http://localhost:3000/api
```

`<sua-entidade-no-plural>` e `users` vazios é o resultado **correto e esperado** hoje — o que estamos validando é o **formato** da resposta, não o conteúdo (ainda não há banco de dados).

Checklist desta etapa:

- [ ] `curl .../api/search?q=teste` responde no formato esperado, com o nome certo do campo
- [ ] `curl .../api/search` (sem `q`) responde com `query: ""`
- [ ] `curl .../api` continua respondendo normalmente

PARTE B — Frontend: a camada de comunicação com a sua API

9. Contextualização rápida

Se cada tela nova do seu projeto (Login, Feed, Upload, Perfil...) escrevesse sua própria chamada `fetch`, resolvendo sozinha o endereço da API e tratando erro do seu próprio jeito, ao final do semestre a URL da sua API estaria espalhada em dezenas de arquivos, cada um tratando erro de um jeito diferente. A solução de hoje: uma camada `services/`, com um arquivo por módulo de domínio da sua API — o mesmo espelhamento 1-para-1 com `modules/user`, `modules/<sua-entidade>`, `modules/search` do seu back-end que já vínhamos usando na estrutura de pastas desde a Aula 01. A partir de hoje, **nenhum componente Vue chama `fetch` ou `axios` diretamente**.

10. Etapa 1 — Instância única do Axios

Crie `src/services/api.js` (o código é o mesmo, independentemente do seu projeto):

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
})

export default api
```

Este é o **único** arquivo do seu projeto que importa `axios` diretamente — todo o resto vai importar `api` a partir daqui. `baseURL` vem do `.env` que você já configurou na Aula 01 (`VITE_API_URL=http://localhost:<porta-da-sua-api>/api`), e é isso que permite trocar o ambiente inteiro (local → produção) mudando uma única linha, sem tocar em nenhum service.

Checklist desta etapa:

- [ ] `src/services/api.js` criado com `axios.create(...)` usando `VITE_API_URL`

11. Etapa 2 — Interceptor de resposta (tratamento de erro centralizado)

Edite `src/services/api.js` , acrescentando o interceptor logo após a criação da instância:

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
})

api.interceptors.response.use(
  (response) => response.data,
  (error) => {
    if (error.response) {
      // A API respondeu, mas com um status de erro (4xx ou 5xx)
      const apiError = error.response.data
      return Promise.reject({
        message: apiError.message || 'Ocorreu um erro na requisição.',
        errors: apiError.errors || [],
        status: error.response.status,
      })
    } else if (error.request) {
      // A requisição foi enviada, mas nenhuma resposta chegou (API fora do ar, sem rede)
```

```
      return Promise.reject({
        message: 'Não foi possível se conectar ao servidor. Verifique sua conexão ou tente
        errors: [],
        status: null,
      })
    } else {
      // Erro ao montar a própria requisição (configuração inválida, por exemplo)
      return Promise.reject({
        message: 'Erro inesperado ao preparar a requisição.',
        errors: [],
        status: null,
      })
    }
  }
)

export default api
```

Resumo dos três ramos (escreva, no topo do seu próprio arquivo, um comentário com suas palavras explicando cada um — essa explicação faz parte da entrega, veja a Seção 15):

- Sucesso (`(response) => response.data`) → "descasca" um nível: cada chamada feita através de `api` já recebe diretamente `{ success, message, data }` , sem precisar escrever `response.data` toda vez.
- `error.response` → a API respondeu, mas com status de erro (`400` , `401` , `404` ...). O erro já formatado pelo seu `apiResponse.js` (Parte A da Aula 01) é reaproveitado.
- `error.request` → a requisição foi enviada, mas **nenhuma resposta chegou** (API fora do ar, ou sem conexão) — não existe `error.response.data` para ler, por isso a mensagem é genérica.
- `else` → erro raro, na própria montagem da requisição.

Em qualquer um dos três casos, o resultado é sempre um `Promise.reject` no mesmo formato `{ message, errors, status }` — é essa uniformidade que faz o tratamento de erro deixar de ser "um jeito por tela" e virar um único padrão para o projeto inteiro.

Checklist desta etapa:

- [ ] Interceptor de resposta adicionado a `api.js`
- [ ] Comentário no topo do arquivo, com suas palavras, explicando os três ramos

12. Etapa 3 — Criando os primeiros services

Crie `src/services/authService.js` (código idêntico para qualquer projeto — essas funções ainda **não são usadas** hoje; a tela de Registro/Login chega em aulas futuras):

```
import api from './api'

export function register(payload) {
  return api.post('/register', payload)
}

export function login(payload) {
  return api.post('/login', payload)
}

export function logout() {
  return api.post('/logout')
}
```

Crie `src/services/searchService.js`:

```
import api from './api'

export function search(query) {
  return api.get('/search', {
    params: { q: query },
  })
}
```

> ⚠️ O nome do parâmetro, `q`, **precisa ser exatamente esse** — é o nome que o `searchController.js` da sua própria API (Etapa 2 da Parte A) já espera em `req.query.q`.

Crie `src/services/systemService.js`:

```
import api from './api'

export function getApiStatus() {
  return api.get('/')
}
```

Checklist desta etapa:

- [ ] `authService.js` criado com `register`, `login`, `logout` (ainda sem uso)
- [ ] `searchService.js` criado, usando `params: { q: query }`
- [ ] `systemService.js` criado

13. Etapa 4 — Refatorando a Landing Page

Edite `src/views/LandingView.vue`, trocando o `fetch` cru da Aula 01 pela chamada através do `systemService`:

```
<script setup>
import { ref, onMounted } from 'vue'
import { getApiStatus } from '../services/systemService'

const apiStatus = ref('verificando...')

onMounted(async () => {
  try {
    const response = await getApiStatus()
    apiStatus.value = response.data.status // "online"
  } catch (error) {
    apiStatus.value = 'offline'
    console.error('Erro ao consultar a API:', error.message)
  }
})
</script>

<template>
  <div class="landing">
    <h1>Bem-vindo ao <!-- nome do seu projeto --></h1>
    <p>Status da API: <strong>{{ apiStatus }}</strong></p>
  </div>
</template>
```

Repare em duas mudanças em relação à Aula 01: não é mais preciso chamar `.json()` manualmente (o Axios já entrega o corpo pronto), e `error.message` agora funciona de forma confiável em **qualquer** tipo de falha, graças ao interceptor da Etapa 2.

Teste: recarregue `http://localhost:5173/` com a sua API rodando → deve continuar aparecendo "Status da API: online", só que agora por trás das cortinas a chamada passa pela camada de serviços.

Checklist desta etapa:

- [ ] `LandingView.vue` usando `getApiStatus()` em vez de `fetch`
- [ ] Página ainda mostra "Status da API: online"

14. Etapa 5 — Validando a camada com a busca

14.1. Teste temporário

Adicione, dentro do mesmo `onMounted` da Landing Page, logo abaixo do `try/catch` do `getApiStatus`:

```
import { search } from '../services/searchService'

// ... dentro do onMounted, após o bloco try/catch do getApiStatus:
```

```
search('a')
  .then((response) => {
    console.log('Busca OK:', response.data)
  })
  .catch((error) => {
    console.error('Erro na busca:', error.message)
  })
```

Abra o Console do navegador (F12) e confira a mensagem `Busca OK:` com um objeto no formato `{ query: 'a', <sua-entidade-no-plural>: [], users: [] }` — listas vazias aqui **não são erro**, é uma busca bem-sucedida que não encontrou nada (ainda não há dados cadastrados).

14.2. Provocando o erro de rede de propósito

Pare a sua API (`Ctrl+C` no terminal dela) e recarregue a Landing Page.

Esperado: tanto `apiStatus` quanto o log da busca caem no ramo de erro, e a mensagem no console é a mesma frase amigável escrita no interceptor (*"Não foi possível se conectar ao servidor..."*) — não um erro técnico bruto do Axios. Religue a API antes de prosseguir.

14.3. Removendo o teste temporário

Depois de validar, **remova** o bloco de teste do `search('a')` — ele cumpriu seu papel; a função `search` continua existindo em `searchService.js`, pronta para ser usada de verdade quando a tela de Busca chegar.

Checklist desta etapa:

- [ ] Console mostrou `Busca OK:` com o objeto esperado, usando o nome certo da sua entidade
- [ ] Erro de rede provocado de propósito caiu na mensagem amigável do interceptor, não num erro técnico
- [ ] Bloco de teste removido da Landing Page ao final

15. Entrega

Envie em resposta a esta tarefa:

1. Dentro da pasta do seu projeto frontend, na subpasta "01 - Docs" já criada na atividade anterior, acrescente:
   - A ficha em markdown, criada na atividade 01, atualizada conforme solicitado na seção 2.1 desta atividade;
   - Crie uma subpasta chamada `atividade02` e armazene dentro dela:
     - Um print de `curl "...api/search?q=teste"` respondendo corretamente;
     - Um print do Console do navegador mostrando `Busca OK:` com o objeto esperado;

    - Um print do erro de rede provocado de propósito (API desligada), mostrando a mensagem amigável do interceptor;
    - O comentário escrito no topo de `api.js` explicando os três ramos do interceptor (tarefa da Etapa 2).
2. Um primeiro arquivo .zip com a pasta compactada do seu projeto de back-end (sem a pasta node_modules), nomeado como --backend.zip (exemplo:PauloManseira-EduStream-backend.zip), agora incluindo o módulo módulo `modules/search/` ;
3. Um segundo arquivo .zip com a pasta compactada do seu projeto de front-end (sem a pasta node_modules), nomeado como --frontend.zip (exemplo: PauloManseira-EduStream-frontend.zip), agora incluindo o `src/services/` ;

> Garanta que você conferiu, executou e marcou todos os checklists desta atividade (Parte A e Parte B).
